27/10/26

Datheon

Machine Learning and Data Science

Task 1:- Multi-class Benchmark Classification Pipeline (Iris/Wine)

- predicting one category from 1 or more categories.
- predict category rather than numerical value.
- sequence of steps (data preprocessing, model training)
- compare diff. models to see how well they perform.

Goal:-
Build end-to-end classification pipelines with Scikit-learn, data normalization, and confusion matrices.

Supervised learning:- Training a model using examples of known ans.

1. Iris Flower
   - Sepal length
   - Sepal width
   - Petal length
   - Petal width.

   based on these measurements, the model predict in which species the flower belong to.
   - * Iris setosa
   - * Iris versicolor
   - * Iris virginica.

Features- I/p variables used to make predictions.

Target - o/p the model predict & is needed too.

To understand models :-

i) Logistic regression.
   ↓ classification.
   * learns patterns in I/p features & predict the probability that an observation belongs to each class.

ii) Decision tree.
   * makes predictions based on sequence of decisions.

Why train 2 models?
→ compare performance.
* Accuracy
* Precision
* F1 score
* Recall
* Confusion matrices.

Step 1:- Open google colab.

* save as "Task_1_Iris_Wine_Classification.ipynb"

Step 2:- Upload csv file.

To check if the upload worked ✓

```
import os
print(os.listdir('/content'))
```

Then, run the cell (▶) button. or ~~shift~~ ctrl + Enter

If ['sample_dataset.csv', 'sample_data'], then uploaded.

Pandas

* is a python library used to load, organize, analyze, and manipulate data.

Dataframe: a table containing rows & columns of data.

csv: a file format used to store tabular data.

Code

```
import pandas as pd
df = pd.read_csv('/content/Iris.csv')  ⇒ reads csv file.
df.head()  ⇒ display 1st 5 rows by default.
```

O/P:- 1st 5 rows of from the dataset.

Step 2:- Understand the dataset.

[Code]

```
print ("Dataset shape:", df.shape)  => eg: (150, 5)
print ("\n column names:")
print (df.columns)
print ("\n Dataset info:")
print(df.info)
df.info
print ("\n Missing values:")
print (df.isnull().sum()) => Counts missing value in each col.
```

Data Exploration :-

is the process of understanding the dataset's structure, columns, data types, and missing values before applying M-L algorithms.

Step 3:- Understand data using statistics

Descriptive Statistics: summarize the main characteristics of a dataset using measures such as:

mean - avg value
std - s.d (measures how spread out values are)
min - smallest value.
count - no. of non-missing values.
25% - 1st quartile
50% - Median
75% - Third quartile
max - largest value.

o/p in this format.

df.describe() => calculates summary stat for numerical cols.

Step 4:- Exploratory Data Analysis (EDA)

⇓

is the process of analyzing a dataset using stat and visualizations to discover patterns, identify relationships and understand the data before building a ML model.

4.1:- Check the no. of flowers in each species

⟹ `df['Species'].value_counts()`

O/P:-

Species	count
setosa	50
versicolor	50
virginica	50

dtype: int64

4.2 Visualize the species distribution.

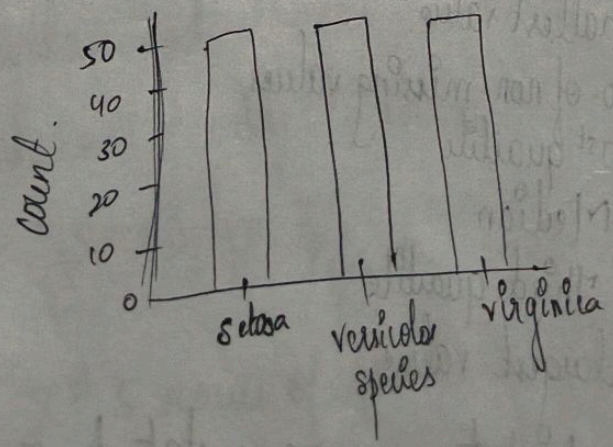
⟹
```
import matplotlib.pyplot as plt
df['species'].value_counts().plot(kind='bar')
plt.title('Distribution of Iris species')
plt.xlabel('species')
plt.ylabel('count')
plt.xticks(rotation=0)
plt.show()
```

⇒ controls x-axis tick marks.

⇓

Keeps labels horizontal.

O/P:-



24/10/26

Steps: Separate features and target.

Features - Numerical: Sepal length, Sepal Width, Petal Length, Petal Width

Species → Target → categorical

Code

```
x = df.drop('species', axis=1)   # remove a specific column; column to remove
y = df['species']                # stores species column in y; work with column not row
print("Features:")
print(x.head())
print("In Target:")
print(y.head())
print(x.shape) → (150, 4)
print(y.shape) → (150,)
```

Feature-target separation
↳ is the process of dividing a dataset into I/P variables (x) and O/P variable (y) that a model predict.

Code

Step6: Split the datasets into training and testing sets

training → used to teach the model.

testing → used to evaluate how well the model predicts the unseen data.

80:20 → split.

Code

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
                                   random_state=42, stratify=y)
print("Training features:", x_train.shape) (120,4)
print("Testing features:", x_test.shape) (30,4)
print("Training targets:", y_train.shape) (120,)
print("Testing targets:", y_test.shape) (30,)
```

test_size=0.2 ⇒ allocates 20% for testing.

random_state=42 → makes random split reproducible

stratify=y → helps preserve the species proportion in both the sets.

150 rows → 120 train, 30 test.

Step 7:- Feature scaling using StandardScaler.

↓

transforms numerical features to a comparable scale so that differences in their magnitudes don't disproportionately affect certain machine learning algorithms.

Standard scaler:

A feature scaling technique that transforms numerical features to have a mean of approximately 0 and a standard deviation of approximately 1, based on the data used for it.

Code

```
from sklearn.preprocessing import StandardScaler → creates a scaler.
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train) → learn mean and s.d of training data.
x_test_scaled = scaler.transform(x_test) → scales testing data using training stat.
print("Scaled training data shape:", x_train_scaled.shape)
print("Scaled testing data shape:", x_test_scaled.shape)
```

o/p:- (120,4)
(30,4)

Step 8:- Train Logistic Regression Model

↓

- is a supervised M-L algorithm used for classification.

Q What happens during training?

→ The model learns relationships btw flower measurements and their species labels.

* It uses those relationships to classify new flowers.

Code

```
from sklearn.linear_model import LogisticRegression   # creates classification model
lr_model = LogisticRegression(max_iter=200)           # Allows upto 200 iteration for training
lr_model.fit(x_train_scaled, y_train)                 # train model using training data
print("Logistic Regression model trained successfully!")   # o/p
```

Step 9:- Make Predictions. (Testing)

Code

```
y_pred_lr = lr_model.predict(x_test_scaled)   # train model predicts
print("Actual species:")
print(y_test.to_numpy())                      # contains actual species
print("\n Predicted species:")
print(y_pred_lr)                              # contains model predicted species
```

Step 10: Evaluate the Logistic Regression Model.

Accuracy: is the proportion of correct predictions made by a classification model out of all predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Manual calc

$$\text{Acc} = \frac{\text{Correct pred}}{\text{Total pred}} \times 100$$

$$= \frac{28}{30} \times 100$$

$$= 93.33\%$$

93.33
3 | 280
27↓
18
18
10
9
10
9
1

Code

```
from sklearn.metrics import accuracy_score
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print("Logistic Regression Accuracy:", round(accuracy_lr*100, 2), "%")
```

→ compares actual & predicted species.

displays two decimal places

stores the acc as decimal

O/P: 93.33%

Step 11: Generate the Classification Report.

Metric	Meaning
Precision	- How many predictions were correct.
Recall	- How many actual species were correctly identified.
F1 score	- Balance btw precision & recall.
Support	- No. of actual test samples belonging to each species.

Code

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_lr))
```

O/P:- gives precision, recall, f1-score and support of setosa, versicolor, virginica, accuracy, macro avg, weighted avg.

Step 12:- Train a Decision Tree classifier.

⇓

is a supervised ML algorithm that makes predictions by following a series of decision rules.

Code

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(x_train, y_train)
print("Decision Tree model trained successfully!")
```

o/p:-

* We use x_train instead of x_train_scaled is because decision tree don't require feature training, it can trained by og. numerical values.

Make predictions.

Code

```
y_pred_dt = dt_model.predict(x_test)
print("Actual species:")
print(y_test.to_numpy())
print("\n Predicted species:")
print(y_pred_dt)
```

o/p:- same as step 9 code o/p.

* Predicts all 30 samples.

Calculate decision Tree accuracy → Classification report

Same code as in Step 10

just change lr to dt.

o/p is same too 93.33%.

same as step 11

o/p is same too.

Step 13:- Compare Both Models.

1) Create accuracy comparison.

Code

```
import pandas as pd
results = pd.DataFrame({'Model': ['Logistic Regression', 'Decision Tree'],
                        'Accuracy': [accuracy_lr, accuracy_dt]})
results['Accuracy (%)'] = (results['Accuracy'] * 100).round(2)
print(results[['Model', 'Accuracy (%)']])
```

Model comparison: evaluates diff. M.L algorithms using the same test dataset and performance metrics.

O/P:-

	Model	Accuracy (%)
0	Logistic Regression	93.33
1	Decision Tree	93.33.

Step 14:- Plot the Confusion Matrix Heatmaps.

⇓

helps us understand which species the model predicts correctly and which species it confuses with another.

Code

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
cm_lr = confusion_matrix(y_test, y_pred_lr)
```

```
plt.figure(figsize=(6,4))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues', xticklabels=lr_model.
            classes_, yticklabels=lr_model.classes_)
plt.title('Logistic Regression - Confusion Matrix')
plt.xlabel('Predicted Species')
plt.ylabel('Actual Species')
plt.show()
```

Displays counts inside cell

Displays count as integers

use shades of blue

O/P:- Rows - Actual species
Columns - Predicted "
Diagonal cells - Correct predictions
off-diagonal cells - Incorrect "

Box

Decision Tree Confusion Matrix

Code

```
cm_dt = confusion_matrix(y_test, y_pred_dt)
plt.figure(figsize=(6,4))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Greens', xticklabels=
            dt_model.classes_, yticklabels=dt_model.classes_)
plt.title('Decision Tree - Confusion Matrix')
plt.xlabel('Predicted Species')
plt.ylabel('Actual Species')
plt.show()
```

O/P:- Same as above prog's, O/P.